AI-Assisted Legacy Modernization

Pilot report and proposal: a COBOL mainframe application translated to Rust with Claude Code

Illia Burov

November 2026

In one slide

What we did

- Took a complete mainframe application: a credit-card system in **COBOL**, with online screens, nightly batch jobs and mainframe data files
- Had an AI coding agent, **Claude Code**, translate it into **Rust**, a modern, safe, fast language
- Required **proof**: the old and new programs run on the same data and must produce the same output

What we got

- All **30 programs** translated, working and tested
- **Identical results**: 40 of 40 output comparisons against the original COBOL
- **41 of 41** test cases passed
- **12 defects** found in the original, documented, not silently changed
- About **2 hours** of AI working time, with one engineer supervising

Recommendation: run a second pilot on one of *our own* legacy systems, using the same method.

Why legacy code is a business risk

- Core business rules live in decades-old COBOL, PL/I or RPG code
- The people who wrote it are retiring; few new engineers learn these languages
- Mainframe capacity and licences are a large, fixed cost
- Manual rewrites are slow and risky: requirements are buried in the code, and small differences in rounding, dates or edge cases break the business

What a good solution must do

1. Understand the old code, *including its quirks*
2. Produce maintainable code in a modern language
3. **Prove** the new system behaves like the old one
4. Keep people in control of scope, review and release
5. Be fast and repeatable across many systems

The original application: what CardDemo is for

CardDemo is the core system of a **credit-card issuer**, the kind a bank runs on its mainframe. It keeps track of who the customers are, which cards and accounts they hold, what they spent, and what they owe.

Who uses it

- **Bank staff** at their desks, through green-screen terminals: look up and update customer accounts and cards, enter transactions, take bill payments, request reports
- **Administrators**: manage which staff may sign on (user ids, passwords, roles)
- **The nightly batch**: unattended programs that process the day's card activity

What it holds

- 50 customers, each with an account and a credit card (sample data)
- Account balances, credit limits, cash limits, billing-cycle totals
- Every card transaction: purchases, returns, payments, interest
- Transaction types and categories, interest-rate groups
- Staff user ids and roles

AWS published it as a realistic test case for mainframe modernization: it uses the same technology (COBOL, CICS screens, VSAM files, JCL batch jobs) as real banking systems.

A day in the life of the system

During the day: online screens

- **Sign on** and see a menu that depends on your role
- **Accounts:** view an account with its customer, change limits, addresses, phone numbers
- **Cards:** list cards, view details, block a card or change the name and expiry date
- **Transactions:** browse, view, add a transaction manually
- **Bill payment:** pay off an account's balance in full
- **Reports:** request a transaction report for a month, a year or any date range
- **User administration:** add, change, delete staff users

At night: the batch cycle

1. **Posting:** check each of the day's card transactions (valid card? over the limit? expired?), post the good ones to the accounts, put the rest on a reject list
2. **Interest:** calculate monthly interest per account and category, add it as transactions
3. **Combine:** merge the day's and the interest transactions into the master file
4. **Statements:** one statement per customer, as a printable text and as a web page
5. **Reports:** transaction report with page, account and grand totals
6. **Migration export / import** of all data for another branch

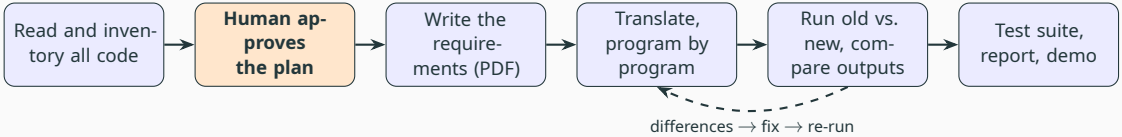
What was translated

	Original (COBOL, mainframe)	Translation (Rust)
Online screens	17 programs (IBM CICS), 17 screen layouts	17 modules + terminal screen engine
Nightly batch	13 programs + JCL job scripts	13 modules + job runner
Data	VSAM files, EBCDIC encoding, packed decimals	Embedded database, same record layouts, same bytes
Size	≈ 27,000 lines	≈ 10,800 lines (incl. generated code)

The application is **AWS CardDemo**, a public sample built to represent real mainframe banking systems. The original code is unchanged in the repository (`cobol/`), next to the translation (`rust/`).

{CardDemo © Amazon.com, Inc. or its affiliates, used under the Apache License 2.0. The Rust translation is a derived work by Illia Burov. This pilot is not affiliated with or endorsed by Amazon Web Services.}

How the AI worked: plan first, prove at the end



- The AI asked what to include before writing any code. A person chose: base application, terminal screens, embedded database, keep the original's behaviour exactly.
- Every phase ended in a saved, reviewable step (18+ commits) with a written log of decisions.
- Deliverables were generated as documents for people: requirements, test suite, demo guide, this report.

Results at a glance

Measure	Result
Programs translated	30 / 30
Output comparisons vs. COBOL	40 / 40 identical
Manual test cases	41 / 41 passed
Automated test suites	20, all passing
Defects found in the original	12
AI working time	≈ 1.9 h

Documents produced by the AI

- Requirements specification: 26 pages, 165 functional requirements
- Manual test suite with execution log
- Live demo guide for non-technical audiences
- Progress report with time and token figures
- `CLAUDE.md`: the rulebook for future AI work on this code

Proof: the old and new systems side by side

Nightly job	What is compared	Result
Daily posting	updated accounts, balances, transactions, rejected items	✓ identical
Interest calculation	accounts, generated interest transactions	✓ identical
Transaction report	519-line printed report	✓ identical
Customer statements	50 statements, text and web version (7,900 lines)	✓ identical
Data export / import	migration file, re-imported files, error file	✓ identical
File print utilities	5 programs incl. packed-decimal outputs	✓ identical

The original COBOL is compiled with a free COBOL compiler and run **on the same data, with the same clock**. Anyone can repeat this check with one command (see the live demo).

What the AI found that people had missed

In the original system (kept, documented as *known defects*)

- Interest for the last account is never saved
- The transaction report counts the last amount twice
- Updating a card resets its security code to 000
- Two programs **do not compile** as shipped
- Saving an account moves its group id into the ZIP-code field
- … 12 in total

In its own translation (found and fixed by the comparison)

- 9 subtle differences, e.g. what is in unused bytes of a record, or a message cut at 75 instead of 78 characters
- None were visible in normal tests. All were caught by **running the original**
- 8 test-case descriptions were also corrected against the original code
- An AI **code review** (scoped by `CLAUDE.md` to the Rust code) then found 10 more issues, all fixed with tests

Lesson: the value is not only in writing code fast, but in *verifying* it automatically.

CLAUDE.md: the project rulebook for the AI

`CLAUDE.md` is a plain-text file in the repository. Claude Code reads it at the start of **every** session. It turns one person's experience into standing instructions for every future AI session:

- **Scope:** what the AI may change and what is read-only
- **How to build, test and prove** the system
- **Review policy:** what to review and what never to touch
- **Lessons learned:** rules found the hard way, so no session repeats a mistake

```
## Code review
When asked to review code, review only rust/.
- cobol/ is the original source and the
  specification. Never report findings
  against it and never propose changes.
- A behaviour that differs from the COBOL
  is a Rust bug, even if the COBOL looks
  wrong. Known defects (KD-nn) are kept
  on purpose; do not report them.

## Translation rules (learned the hard way)
- Storage without VALUE is binary zeros ...
```

Why it matters for scale: the same rulebook makes many sessions, teams and systems behave consistently. It is reviewed and versioned like code.

Human oversight built into the workflow

- **Plan approval:** no code until a person accepted the plan and the scope
- **Checkpoints:** each step is a separate commit that can be reviewed or undone; nothing is published without being asked
- **Budget limits:** the AI was told to stop at 85% of the usage quota and write a hand-over note. Work resumed the next day exactly where it stopped
- **Memory and logs:** decisions, findings and metrics are written down, not kept in someone's head

Claude Code Remote Control

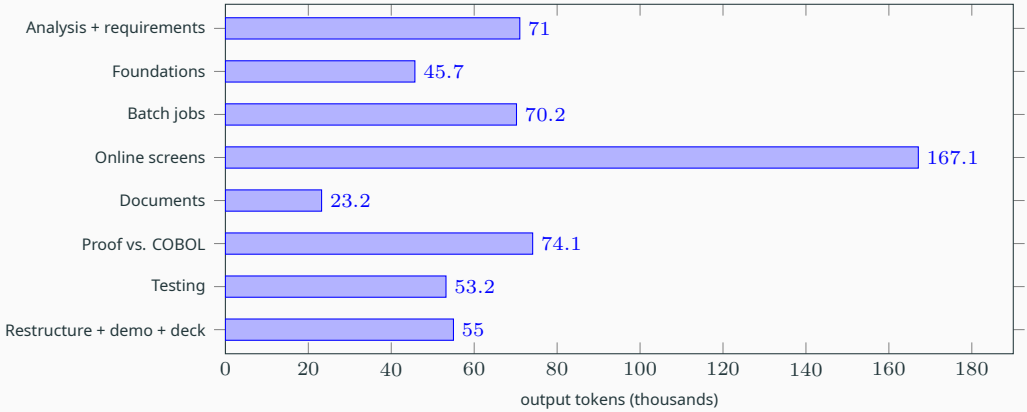
- The work runs on the company workstation; the code stays there
- From the **Claude mobile app**, the supervising engineer follows the session in progress, answers the AI's questions and approves steps while away from the desk
- Long translation runs no longer need someone sitting at the screen

Time and tokens

Session	Work	AI time	Output tokens
1	Analysis, requirements, translation of all 30 programs, test suite, report	1.1 h	367k
2	Proof against the original COBOL, test execution, fixes	0.5 h	137k
3	Repository restructure, demo tooling and guide, `CLAUDE.md`, this presentation	0.3 h	55k
Total		≈ **1.9 h**	≈ **0.56M**

- Measured from the Claude Code session logs (time with pauses over 10 minutes removed).
- Token cost depends on the subscription or API price list in use.
- The engineer’s own time went into decisions and reviews, not into typing code.

Where the effort went



The online screens were the largest part: 17 interactive programs with many validation rules each.

Risks and mitigations

Risk	Mitigation in this method
AI produces plausible but wrong code	Run old and new on the same data; outputs must be identical
Hidden rules in old code are lost	Requirements are written first, from the code, and reviewed
Original defects get “fixed” silently	Defects are kept and documented; fixing them is a business decision
Parts that cannot run outside the mainframe (online CICS screens)	Screens drawn from the original layouts; 41-case test suite; next: comparison on a test mainframe
Source code confidentiality	Code stays on company machines; only what is sent to the model leaves them, under the company’s agreement with the vendor
Over-reliance on one tool	Output is ordinary Rust code plus documents; any team can maintain it

From pilot to programme

Next pilot (4–6 weeks)

1. Pick one of our legacy systems: bounded, with test data
2. Write its `CLAUDE.md`: scope, target stack, review rules
3. Set up the comparison harness: old vs. new on the same data
4. Translate, verify, document, using the same steps as this pilot
5. Review with the system owners; decide on go-live criteria

What we need

- One engineer who knows the system, part time
- Access to the code and representative test data
- Claude Code licences for the team
- A test environment for the original system (for online comparison)

Success criteria: identical outputs on agreed test data, test suite passed, owners sign off.

Live demo

```
python3 demo/demo.py cobol      # the original COBOL programs
python3 demo/demo.py rust       # the Rust translation
python3 demo/demo.py compare    # → IDENTICAL, file by file
python3 demo/demo.py online     # the online screens
```

Step-by-step guide for presenters and non-technical audiences: `docs/DEMO.md`.

Questions: Illia Burov

Appendix

Technical architecture

Component	Replaces	Lines
`cobol`	COBOL data rules: decimals, EBCDIC; records generated from copybooks	1,677
`vsam`	VSAM indexed files, on SQLite	556
`cics`	CICS screens (from the original BMS files), transfers, file control	833
`online`	the 17 online programs, one function per COBOL paragraph	4,283
`batch`	the 13 batch programs and JCL jobs	1,886
`carddemo`	the application: terminal screens, job runner	440

Verification: `rust/difftest/run.py` (GnuCOBOL comparison), `cargo test` (20 suites, incl. the 41 manual cases as scripted terminal sessions and multi-user tests).

Known defects of the original (selection)

ID	Where	Effect
KD-01	Interest calculation	the last account’s interest is never written back
KD-02/03	Transaction report	last amount counted twice; no final account total
KD-07	Card update screen	the card security code is reset to 000
KD-08	Account update screen	the “blank phone number” test checks the area code twice
KD-10	User / transaction lists	searching past the last record crashes the transaction
KD-12	Export / import	the programs do not compile as shipped
KD-13	Account update screen	the group id is saved into the ZIP-code field

All 12 are in the requirements specification, with the exact behaviour that the translation reproduces.